

WikForge 项目目录结构与文件功能说明

项目路径：`C:\Users\zxsoul\Desktop\面试\agent应用开发\wikforge-main`

项目定位：**企业知识库 + RAG 问答全栈系统**

技术栈：FastAPI 后端 + Next.js 前端 + Celery 异步流水线 + PostgreSQL / OpenSearch / Qdrant / Redis / MinIO + LiteLLM

根目录

wikforge-main/

- └─ README.md # 项目说明
- └─ Makefile # 常用命令封装（启动/测试/迁移）
- └─ pyproject.toml # 根级 Python 工具配置
- └─ docker-compose.yml # 全套基础设施编排（PG/Redis/OpenSearch/Qdrant/MinIO/LiteLLM）
- └─ docker-compose.test.yml # 测试环境编排
- └─ .env / .env.example # 环境变量（模型 key、各组件连接串）
- └─ docs/
 - └─ ROADMAP.md # 路线图
 - └─ deploy.md # 部署文档
 - └─ plugins/parser-plugin-development.md # 解析器插件开发指南
- └─ litellm/config.yaml # LiteLLM Proxy 的配置（模型路由表）
- └─ opensearch/Dockerfile # 定制 OpenSearch 镜像（装 IK 中文分词插件）
- └─ scripts/ # 运维脚本：备份、冒烟测试、compose 校验、初始化 LiteLLM 库

backend/ —— FastAPI 后端（重点学习区）

app/main.py + app/core/ —— 应用骨架与基础设施

app/main.py # FastAPI 入口：注册路由、中间件、异常处理

app/core/

- └─ config.py # 全局配置（Pydantic Settings，含 LITELLM_MODEL/LLM_TIMEOUT 等）
- └─ database.py # SQLAlchemy 2.0 异步引擎与会话管理
- └─ redis.py # Redis 客户端（会话缓存、权限缓存）
- └─ opensearch.py # OpenSearch 客户端 + chunks 索引管理（关键词/BM25 检索）
- └─ qdrant.py # Qdrant 向量库客户端 + collection 管理（语义检索）
- └─ minio.py # MinIO（S3 兼容）对象存储客户端，存原始文件
- └─ security.py # 密码哈希、JWT 签发/解析
- └─ celery_app.py # Celery 应用配置（异步任务）
- └─ exceptions.py # 自定义异常 + 全局异常处理器
- └─ logging.py # 日志配置

app/api/ —— HTTP 路由层（薄层，只做参数校验和转发）

```
|— auth.py                # 注册/登录/刷新 Token/OIDC 回调
|— documents.py           # 空间/文件夹/标签/文档的 CRUD
|— search.py             # 搜索接口
|— rag.py                # RAG 问答接口
|— qa.py                 # 问答（会话历史相关）
|— feedback.py           # 搜索结果用户反馈
|— permissions.py        # 权限管理
|— ik_dict.py            # IK 分词器远程词库分发（让 openSearch 同步领域词典）
|— health.py             # 健康检查
|— admin_*.py            # 管理后台：用户、监控、词典、文档 Profile、审核队列、解析器配置
                        #   admin_users.py      管理员用户管理
                        #   admin_monitoring.py  系统监控
                        #   admin_dictionaries.py 领域词典管理
                        #   admin_profiles.py   文档 Profile 管理
                        #   admin_reviews.py    文档审核队列管理
                        #   admin_universal_parser.py Universal Parser 运行期模型配置
```

app/models/ —— 数据库表模型（SQLAlchemy）

```
|— base.py                # declarative base 与通用 mixin
|— user.py               # 用户
|— space.py / folder.py  # 空间（顶级组织单元）/ 文件夹层级
|— document.py / document_tag.py # 文档元数据与处理状态 / 标签
|— chat.py              # 问答会话与消息记录
|— permission.py         # ABAC 权限模型
|— document_profile.py / profile_version.py # 文档解析策略"档案"及其版本历史
|— document_review.py    # 解析质量人工审核 workflow
|— domain_dictionary.py  # 领域词典（行业术语）
|— parser_plugin_config.py # 解析器插件注册配置
|— search_feedback.py    # 搜索质量反馈
```

app/services/ —— 业务核心（最该精读的部分）

RAG 链路（对应 LangChain 知识体系）：

```
|— llm_gateway.py        # ★ LLM 网关：基于 LiteLLM 的多模型统一调用
|                        #   （供应商解耦、双层超时、错误码分类、脱敏日志、
|                        #   文本/流式/多模态三种调用、依赖注入）
|— rag_engine.py         # ★ RAG 引擎：对话式问答主编排（≈ 手写 Chain）
|— rag_service.py        # RAG 问答核心服务（非流式/流式生成、引用组装）
|— query_enhancer.py     # 查询增强总控：改写+HyDE+子查询分解，带超时降级
|— query_rewriter.py     # 查询改写（≈ LangChain query rewriting chain）
|— hyde_service.py       # HyDE：让 LLM 生成假想答案去检索
|— subquery_decomposer.py # 复杂问题拆成多个子查询
|— search_service.py     # ★ 复合检索：多路召回 + RRF 融合 + Cross-Encoder 精排
|                        #   （≈ Retriever + Rerank）
|— embedding_service.py  # 稠密+稀疏向量生成（≈ Embeddings 接口）
|— conversation_service.py # 会话历史管理（≈ Memory）
```

文档处理流水线（RAG 的"入库"侧，对应 LangChain Loader/Splitter）：

```
├─ parsers/                                # 解析器插件体系
│   ├── base.py                            # 插件协议 + 中间表示（IR）数据结构
│   ├── registry.py                        # 插件注册/选择/热更新
│   ├── pdf_parser.py                     # PDF（Marker 库）
│   ├── docx_parser.py                   # Word（python-docx）
│   ├── pptx_parser.py                   # PPT（python-pptx）
│   ├── html_parser.py                   # HTML（trafilatura）
│   └── text_parser.py                   # Markdown/纯文本
├─ document_processor.py                 # 清洗、去噪、标题识别、转 Markdown
├─ chunker.py                           # 智能分块（按 Profile 策略、保留层级，≈ TextSplitter）
├─ universal_parser.py                   # 兜底解析器：没有匹配 Profile 时用 LLM（多模态）解析
├─ universal_parser_trigger.py           # 兜底解析的触发条件判断
├─ upload_service.py                     # 文件上传/URL 导入/状态管理/重试
├─ indexing_service.py                   # 双写索引：Qdrant + OpenSearch，失败回滚
├─ quality_scorer.py                     # 解析质量自动评分
├─ review_queue.py                       # 低质量文档进入人工审核队列
├─ profile_matcher.py                     # 自动为文档匹配解析 Profile
├─ profile_candidate_service.py           # 候选 Document Profile 持久化
├─ profile_version_service.py             # Profile 版本快照与变更人查询
├─ dictionary_service.py / domain_dictionary.py # 领域词典管理 + 同步到 IK 分词
├─ feedback_service.py                   # 搜索反馈收集、聚合、优化建议
├─ permission_service.py                 # ABAC 权限判定（检索时过滤无权文档）
├─ document_service.py                   # 文档/空间/文件夹业务逻辑
└─ auth_service.py                       # 认证业务：登录锁定、JWT、OIDC
```

app/tasks/ —— Celery 异步任务

```
├─ pipeline.py                           # 文档处理流水线任务链：解析→清洗→分块→向量化→索引
├─ permission_tasks.py                   # 权限变更异步同步到 Qdrant（保证检索过滤即时生效）
└─ watchdog.py                           # 看门狗：捞回卡在处理中的文档
```

其余

```
├─ alembic/                              # 数据库迁移（versions/ 里是建表脚本）
├─ app/scripts/init_db.py                # 初始化数据库
├─ eval/                                  # ★ RAG 效果评估
│   ├── metrics.py                       # 评估指标
│   ├── qa_pairs.json                    # 测试问答对
│   └── run_eval.py                       # 评估执行入口
├─ tests/                                # 100+ 测试文件，单元测试 + integration/（端到端），
│   └── test_llm_gateway.py               # 等可直接当文档读
├─ requirements.txt                       # 依赖清单（litellm、langchain-text-splitters 等）
├─ Dockerfile / pyproject.toml / alembic.ini
└─ docs/plugin-development.md             # 插件开发文档
```

frontend/ —— Next.js 前端（了解即可）

src/app/	# 路由页面
└─ (auth)/	# 登录/注册/OIDC 回调
└─ (main)/	# 主界面:
	# chat/ 问答界面
	# documents/ 文档管理
	# search/ 搜索
	# dashboard/ 仪表盘
	# spaces/ 空间管理
	# settings/ 设置
	# admin/ 管理后台 (用户/权限/词典/Profile/审核/监控/LLM 配置)
src/components/	# 按域组织:
	# chat/ 消息气泡、引用链接、会话列表、聊天界面
	# documents/ 上传、文件夹树、标签、删除/移动对话框
	# admin/ LLM 配置、监控面板、权限配置、用户/空间管理
	# auth/ 登录/注册表单、路由守卫、OIDC 按钮
	# layout/ 侧边栏、顶栏、全局搜索
	# ui/ 基础组件 (button/card/dialog/toast 等)
src/stores/	# Zustand 状态: auth、chat、search、sidebar
src/lib/api-client.ts	# 后端 API 封装
src/hooks/	# use-debounce、use-media-query
e2e/	# Playwright 端到端测试
其他:	next.config.mjs / tailwind.config.ts / nginx.conf / Dockerfile

建议的阅读顺序（面试导向）

1. `docker-compose.yml` + `app/core/config.py` —— 先搞清系统有哪些组件
2. `app/tasks/pipeline.py` —— 文档入库主线（对应 LangChain 的 Loader→Splitter→Embedding→VectorStore）
3. `rag_engine.py` → `query_enhancer.py` → `search_service.py` → `rag_service.py` —— 问答主线（对应 Retriever→Rerank→Chain）
4. `llm_gateway.py` + `tests/test_llm_gateway.py` —— LLM 网关，测试文件可辅助确认理解
5. `eval/` —— 企业项目如何做 RAG 效果评估，面试加分项

附：与 LangChain 概念对照表

LangChain 概念	本项目对应物
<code>ChatOpenAI</code> / <code>.invoke()</code>	<code>LLMGateway.complete()</code> （底层 LiteLLM）
<code>.stream()</code> 流式输出	<code>LLMGateway.stream()</code> 异步生成器
<code>ChatPromptTemplate</code>	各 service 内直接拼 system prompt 字符串
Retriever / <code>similarity_search</code>	<code>search_service.py</code> （OpenSearch + Qdrant 混合检索）
Reranker	<code>search_service.py</code> 的 Cross-Encoder 精排

LangChain 概念	本项目对应物
<code>RecursiveCharacterTextSplitter</code>	<code>chunker.py</code> (依赖里唯一保留的 langchain-text-splitters)
Embeddings 接口	<code>embedding_service.py</code>
LCEL Chain 编排	<code>rag_engine.py</code> 的 <code>chat()</code> 手写 async 流程
Memory	<code>conversation_service.py</code>
查询改写 / HyDE / 子查询分解	<code>query_rewriter.py</code> / <code>hyde_service.py</code> / <code>subquery_decomposer.py</code>
Document Loader	<code>parsers/</code> 插件体系 + <code>universal_parser.py</code> 兜底